Firewall Log Classification in Data Streams Using Genetic Programming

Renato Calacina Spessotto - 14605824[1]
Jean Patrick Ngandu Mamani - 14712678[2]
Guilherme Torquato - 14575787[3]

[1]ICMC – University of Sao Paulo (USP) – Sao Carlos – SP – Brazil

{renatospessotto, jeanjpnm, gtorquato}@usp.br

Abstract. *Network traffic classification is a critical task in cybersecurity, particularly when dealing with high-speed data streams that evolve over time. This paper presents a classification approach for firewall logs using Genetic Programming (GP) in a One-vs-Rest scheme. The dataset consists of 1,517,735 network sessions captured by a Palo Alto firewall, combining two collection phases separated by a 15-day interval to simulate temporal drift. A proportional temporal split allocates the first 30% of each phase for training and the remaining 70% for testing, evaluated across 50 chronological windows. After preprocessing, which includes frequency encoding of categorical attributes and log-normalization of traffic volume counts, four GP trees are evolved independently, one per class. Two fitness formulations are compared: a baseline using the binary F1 score, and a class-weighted variant that penalizes errors on rare classes in inverse proportion to their frequency. The baseline achieved a weighted F1 of 0.888 on the test stream but failed to detect the two minority classes (F1 of zero for both* drop *and* reset-both*). The weighted-fitness variant recovered these classes, raising macro F1 from 0.475 to 0.587 and* drop *F1 to 0.473, at the cost of a small drop in majority-class performance. Both models remained stable across the concept-drift boundary between the two collection phases, with the weighted variant even improving on the more recent data.*

1. Introduction

The Internet connects billions of devices and generates massive volumes of network traffic every second [Al-amri et al. 2021]. In this context, firewalls represent the first line of defense for corporate networks, inspecting each session and applying one of four possible actions, namely allowing the connection, silently discarding packets (drop), blocking with notification (deny), or abruptly terminating the session (reset-both). Correctly assigning these actions is essential for network security, but the rules governing such decisions are in constant evolution to adapt to new attack vectors [Schindler 2017].

Machine Learning (ML) algorithms have shown significant potential for automating and improving the network traffic classification process [Mitchell 1997, Tu 2019]. Prior work explored approaches such as Support Vector Machines [Ertam and Kaya 2018] and ensemble methods [Sharma et al. 2021] for classifying firewall logs on static datasets, achieving promising results. However, real network environments exhibit data stream characteristics, meaning data arrive continuously at high speed, and statistical distributions may shift over time through a phenomenon known as concept drift [Gama 2010, Gama et al. 2014].

Traditional batch ML, trained on fixed datasets, does not capture this dynamic [Ling et al. 2009]. Furthermore, the scale of network data limits the applicability of many conventional models [Al-amri et al. 2021]. Evolutionary algorithms, and Genetic Programming (GP) in particular [Koza 1992], offer a promising alternative. GP evolves mathematical expressions tailored to the problem without requiring the user to define the model structure in advance, producing compact and interpretable classifiers. Despite these advantages, its adoption for firewall log classification at stream scale remains largely unexplored.

This paper proposes a GP-based classifier for firewall logs under a temporal data stream scenario. The main contributions are fourfold. First, the paper defines a complete stream-oriented preprocessing pipeline with a proportional temporal split that exposes the model to both collection phases during training while preventing any information leakage from the test set. Second, it employs vectorized fitness evaluation over the full training set, making it feasible to use approximately 455,000 instances per fitness call without subsampling. Third, it compares two fitness formulations, a standard binary F1 and a class-weighted variant designed to recover the severely under-represented minority classes. Fourth, it analyzes model behavior separately on each collection phase and across 50 chronological evaluation windows, characterizing stability under the concept drift introduced by the 15-day gap.

The remainder of this paper is organized as follows. Section 2 describes the methodology, covering the dataset, preprocessing, the GP model, the weighted-fitness variant, and the evaluation protocol. Section 3 presents the experimental results. Section 4 discusses the findings and limitations. Section 5 concludes the work.

2. Methodology

2.1. Dataset

The data were collected by a Palo Alto firewall (model 3260) from a higher education institution with over 3,500 employees and 15,000 students. The full dataset is composed of two distinct blocks. The first block, referred to as DS1, contains 1,002,160 instances captured over 17 consecutive days. The second block, DS2, contains 515,575 instances collected 15 days after DS1 over a period of 9 days.

Each instance represents a network session between two devices and has 27 attributes. The target attribute Action assumes four values, with allow accounting for 53.92% of all instances, deny for 39.12%, drop for 6.80%, and reset-both for only 0.16% (2,460 sessions). Figure 1 illustrates the class distribution and the missing-value map.

The two datasets were concatenated in chronological order, forming a single stream of 1,517,735 instances. The DS1-to-DS2 boundary constitutes a potential concept-drift point, since it represents a 15-day temporal gap and a change in the nature of the captured traffic.

2.2. Preprocessing

The preprocessing pipeline was designed to respect the stream nature of the data. All parameters, including encoders, scalers, and imputation values, were estimated exclusively from the training portion and then applied to the test stream, preventing any data leakage.

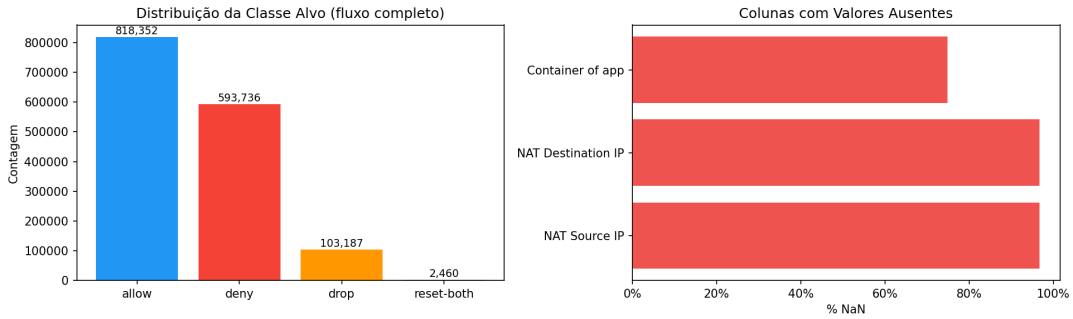


Figure 1. Class distribution in the full stream (left) and percentage of missing values per column (right).

Six columns were removed during the feature selection step. The source and destination address columns were excluded because they act as near-unique identifiers, with approximately 91,000 and 57,000 distinct values respectively. The NAT source IP and NAT destination IP columns were excluded because the vast majority of their values are missing. The container-of-app column was also removed due to high missingness, and the internal phase control variable was dropped. Network zones already encode the network segment information in a compact way, so the address columns offer no additional signal.

The 10 remaining categorical columns, covering application name, network zones, IP protocol, traffic category, source and destination countries, and application taxonomy, were encoded using frequency encoding. Under this scheme, each value is replaced by its relative frequency in the training set. This keeps the GP terminal set small and avoids the dimensionality explosion typical of one-hot encoding.

The seven high-skew numerical columns, namely bytes, bytes sent, bytes received, packets, packets sent, packets received, and elapsed time, received a $\log(1 + x)$ transformation before standardization. The four port-number columns were standardized directly without the log transform. Remaining missing numerical values were replaced by the training median of the corresponding column. The result is a matrix X with 21 numerical features in float32 format and no missing values.

2.3. Proportional Temporal Stream Split

Following the data stream paradigm, the split was strictly temporal with no shuffling. However, rather than taking the first 30% of the concatenated stream, which would place the entire training set inside DS1 and never expose the model to DS2 patterns, the split is taken *proportionally within each phase*. The first 30% of DS1 (300,648 instances) and the first 30% of DS2 (154,672 instances) form the training set, totaling 455,320 instances. The remaining 70% of each phase, 701,512 from DS1 and 360,903 from DS2, form the test stream of 1,062,415 instances.

This design preserves temporal causality within each phase, since every test instance is posterior to the training instances of its own phase, while ensuring that both traffic regimes are represented during training. It also enables a separate evaluation of the DS1 and DS2 test portions to measure the impact of concept drift. Table 1 summarizes the per-class composition of the training set, where the extreme imbalance is evident: reset-both has only 812 training instances.

Table 1. Per-class composition of the training set (455,320 instances).

Class	Train instances	Share
allow	259,273	56.9%
deny	166,496	36.6%
drop	28,739	6.3%
reset-both	812	0.2%

2.4. Genetic Programming Model

Genetic Programming [Koza 1992] is an evolutionary technique that represents solutions as expression trees and evolves them through genetic operators. Crossover exchanges subtrees between two individuals, while mutation randomly replaces a subtree with a newly generated one. The implementation uses the DEAP library [Fortin et al. 2012].

To handle four classes, the One-vs-Rest (OvR) scheme was adopted. Four GP trees are evolved independently, each learning to discriminate its own class against all others. The final prediction is the argmax of the four raw scores.

The function set includes addition, subtraction, multiplication, protected division (which returns 1 when the divisor is near zero), negation, absolute value, and the pairwise maximum and minimum of two arguments. Ephemeral constants are sampled uniformly from the interval $[-1, 1]$. The 21 preprocessed numerical attributes constitute the terminal set, alongside the ephemeral constants.

The fitness of each individual is computed over the full training set at every generation. To make this feasible across 455,000 instances, each tree is compiled into a Python function and applied to the entire training matrix in a single NumPy operation, avoiding per-instance Python loops. Tree height is capped at 6 levels via the staticLimit decorator in DEAP, preventing the bloat that would otherwise degrade both interpretability and runtime. Table 2 summarizes the hyperparameters used.

Table 2. Hyperparameters of the Genetic Programming model.

Parameter	Value	Description
Population size	200	Individuals per class evolution
Generations	40	Number of generations
Crossover prob.	0.70	Probability of crossover
Mutation prob.	0.30	Probability of mutation
Tournament size	5	Tournament selection size
Max tree height	6	Bloat control limit
Elitism	1	Best individual kept per run

2.5. Class-Weighted Fitness Variant

The baseline fitness, the binary F1 score, treats every training instance equally. Under extreme imbalance this leads the OvR argmax to be dominated by the majority-class

trees, so the minority classes are never predicted. To address this, a second variant modifies the fitness of each tree to the *weighted* binary F1, where every instance carries a `sample_weight` inversely proportional to the frequency of its class in the training set.

Formally, the weight of class c is $w_c = N/(K \cdot n_c)$, where N is the training size, K the number of classes, and n_c the count of class c. This yields weights of 0.44 for allow, 0.68 for deny, 3.96 for drop, and 140.18 for reset-both. Since the mean instance weight is 1.0 by construction, a reset-both error weighs about 140 times the average instance, and roughly 320 times an allow instance. The data, terminals, operators, and hyperparameters are identical to the baseline; only the fitness weighting changes.

2.6. Evaluation Protocol

Performance is reported at three granularities. First, global metrics (macro F1, weighted F1, and per-class F1) are computed over the complete test set. Second, the DS1 and DS2 test portions are evaluated separately to expose any impact of the concept-drift boundary. Third, the test stream is divided into 50 chronological windows of approximately equal size, and the macro and weighted F1 are computed per window to reveal temporal stability. Both fitness variants are evaluated under this identical protocol.

3. Experiments

3.1. Experimental Setup

The experiments were conducted on a local machine running Python 3.9, with DEAP 1.4, NumPy 2.0, pandas 2.3, and scikit-learn 1.6. The global random seed was fixed at 42 for reproducibility. The four OvR evolutions were run sequentially, each with a class-specific seed equal to 42 plus the class index. Training the baseline model took approximately 436 seconds (7.3 minutes) and the weighted-fitness model approximately 524 seconds (8.7 minutes).

3.2. Training Fitness per Class

Figure 2 shows the convergence curves of the GP fitness over 40 generations for each class, for both variants.

In the baseline, the allow and deny classes converged quickly, reaching binary F1 values of 0.993 and 0.915 respectively, while drop and reset-both reached lower values of 0.474 and 0.673. Under the weighted fitness, the minority classes attain much higher fitness, 0.811 for drop and 0.979 for reset-both, because errors on their few instances are heavily penalized; the majority classes correspondingly settle at slightly lower weighted fitness (0.913 for allow, 0.802 for deny). In all cases the mean curves remained below the best-individual curves, indicating that population diversity was maintained throughout the evolution.

3.3. Test Stream Performance

Table 3 compares the two variants on the complete test stream, and Figure 3 visualizes the per-class F1.

The baseline achieved a weighted F1 of 0.888 and an overall accuracy of 92.2%, with allow F1 of 0.991 (near-perfect recall) and deny F1 of 0.910. However, it failed completely on the minority classes, yielding F1 of zero for both drop and reset-both.

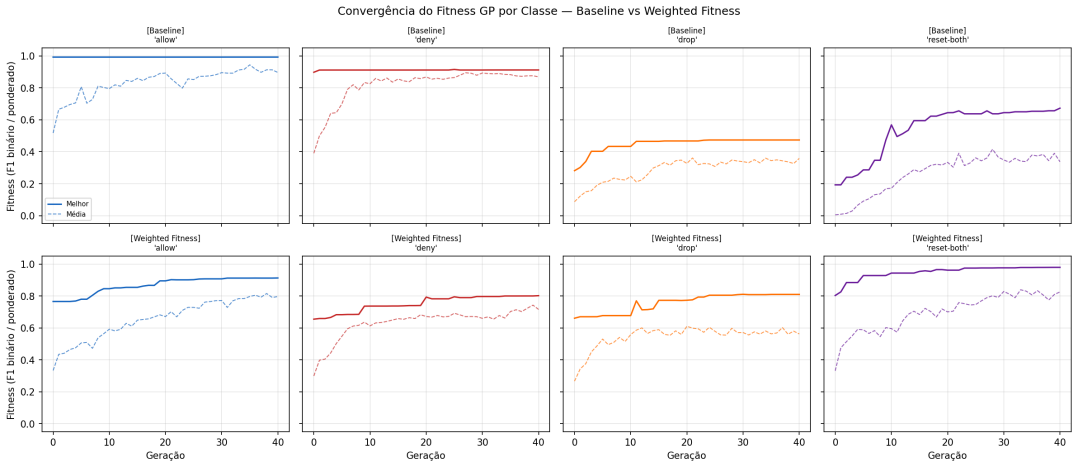


Figure 2. GP fitness convergence curves per class over 40 generations. Top row: baseline (binary F1). Bottom row: weighted fitness (class-weighted binary F1).

Table 3. Per-class F1 and aggregate metrics on the test stream (70% of the full stream, 1,062,415 instances).

Metric	Baseline	Weighted Fitness
allow F1	0.991	0.926
deny F1	0.910	0.886
drop F1	0.000	0.473
reset-both F1	0.000	0.064
Macro F1	0.475	0.587
Weighted F1	0.888	0.877
Accuracy	0.922	0.840

The weighted-fitness variant recovered the minority classes: drop F1 rose to 0.473 (recall 0.68) and reset-both became detectable (F1 0.064, recall 0.97), lifting macro F1 from 0.475 to 0.587. This came at a moderate cost to the majority classes, with allow F1 falling to 0.926 and deny to 0.886, and overall accuracy decreasing to 0.840. The weighted F1 remained essentially unchanged (0.877 versus 0.888), indicating that the model traded a small amount of majority-class precision for a large gain in minority-class recall. Notably, reset-both reaches a recall of 0.97 but a precision of only 0.03, so although nearly all reset-both sessions are now flagged, many deny and drop sessions are over-flagged as reset-both.

Figure 4 shows the normalized confusion matrices for both variants. In the baseline, drop instances are absorbed almost entirely into deny, and reset-both is split between allow and deny. Under the weighted fitness, the diagonal mass for drop and reset-both becomes visible, at the expense of additional off-diagonal spread in the majority classes.

3.4. Concept Drift: DS1 versus DS2

Table 4 reports the macro F1 computed separately on the DS1 and DS2 test portions.

Neither variant degrades on the more recent DS2 data. The baseline is essentially

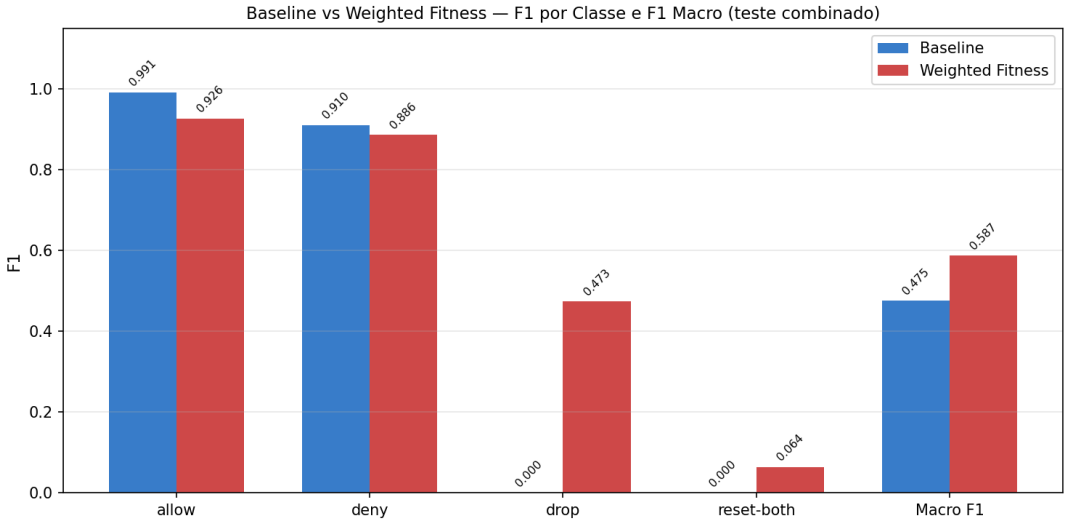


Figure 3. Per-class F1 and macro F1 on the test stream: baseline versus weighted fitness.

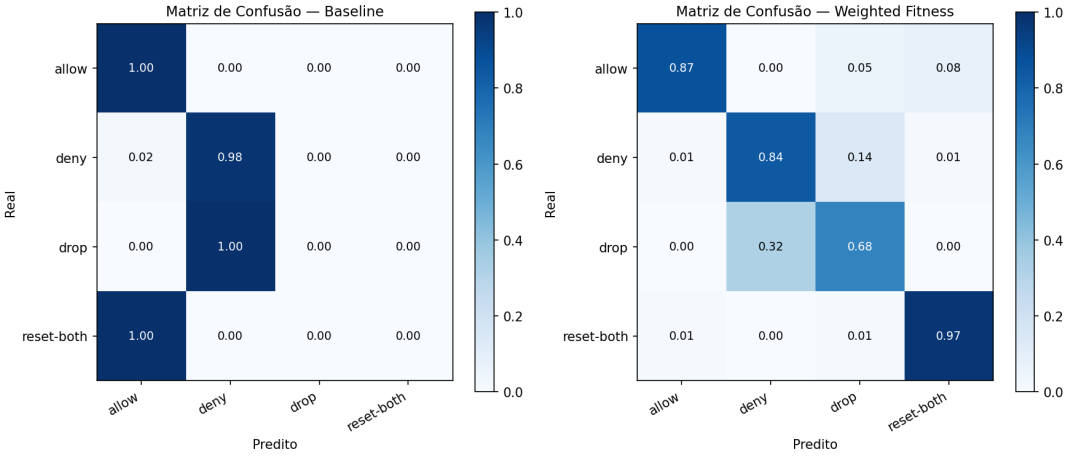


Figure 4. Normalized confusion matrices on the complete test stream: baseline (left) and weighted fitness (right).

flat across the boundary (0.476 to 0.474), and the weighted variant actually improves on DS2 (0.578 to 0.606). Because the proportional split exposes the model to 30% of DS2 during training, the DS2 traffic regime is already represented in the learned expressions, which explains the absence of any drift-induced drop.

3.5. Windowed Evaluation over 50 Windows

Figure 5 shows the macro and weighted F1 curves across the 50 chronological evaluation windows for both variants.

The baseline macro F1 averaged 0.474 ± 0.009 across windows (range 0.450 to 0.487), and the weighted variant averaged 0.582 ± 0.026 (range 0.536 to 0.639). The low variance of both curves indicates stable behavior over time, and crucially there is no abrupt drop at window 33, the DS1-to-DS2 boundary. The weighted variant exhibits slightly higher window-to-window variance, reflecting the more volatile behavior of the

Table 4. Macro F1 on each phase of the test stream.

Variant	DS1 test	DS2 test
Baseline	0.476	0.474
Weighted Fitness	0.578	0.606

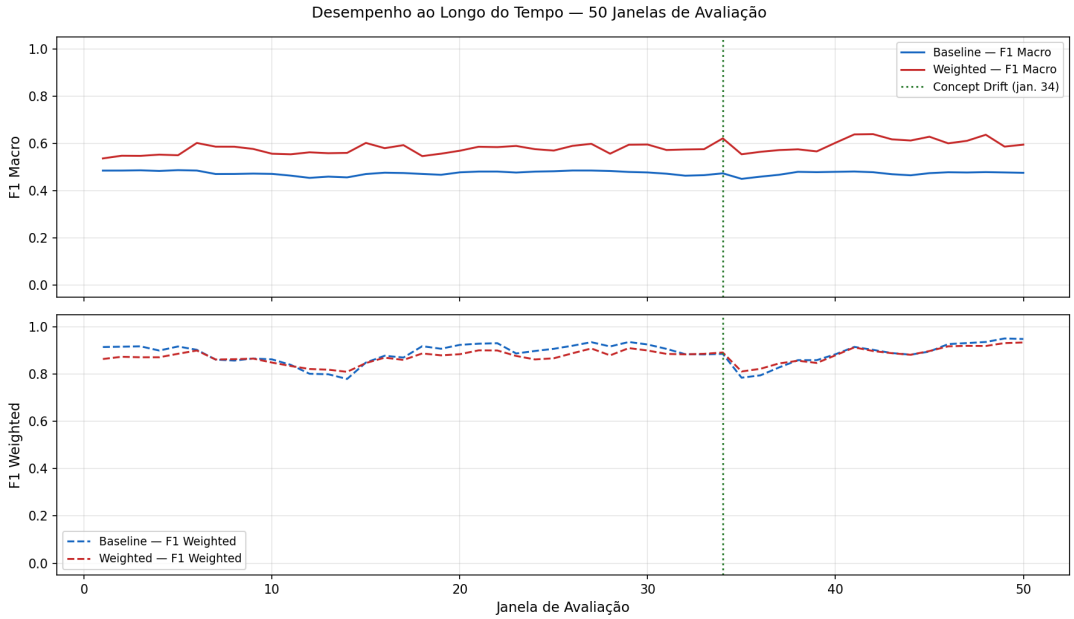


Figure 5. Macro F1 (top) and weighted F1 (bottom) over 50 evaluation windows for both variants. The dotted vertical line at window 33 marks the DS1-to-DS2 boundary, which is the concept-drift point.

minority-class predictions. Figure 6 details the per-class F1 across the same windows, making explicit that drop and reset-both remain at zero throughout for the baseline but stay consistently above zero for the weighted variant.

4. Discussion

4.1. Performance on Majority Classes

Both models demonstrated strong discriminative capacity for the allow and deny classes, which together account for 93% of the full stream. The baseline trees are compact and human-readable: the deny tree, for instance, is essentially a linear combination of the frequency-encoded application and source-zone attributes plus a protocol-by-port interaction term. This illustrates the capacity of Genetic Programming to find interpretable, low-complexity classifiers from high-dimensional data without any prior feature-engineering guidance.

4.2. Recovering the Minority Classes

The inability of the baseline to detect drop and reset-both is its main limitation. Although the baseline trees for these classes achieved binary F1 values of 0.47 and 0.67 on the training set, their raw scores never surpass those of the allow and deny trees in the argmax step.

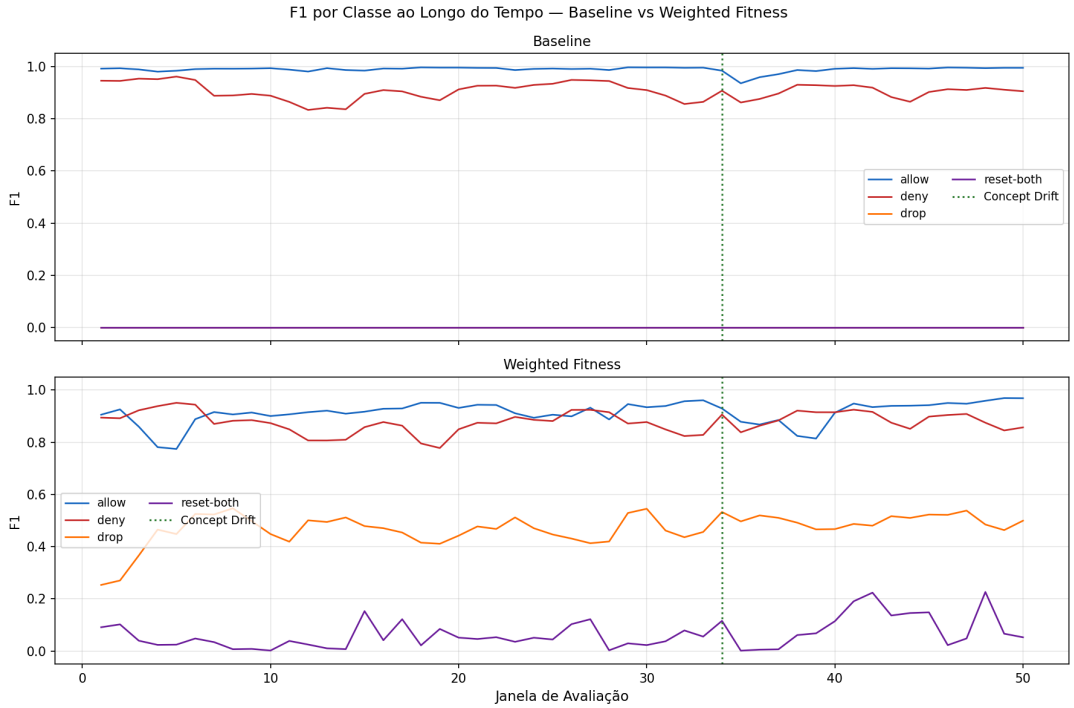


Figure 6. Per-class F1 over 50 chronological evaluation windows: baseline (top) and weighted fitness (bottom).

The class-weighted fitness directly attacks this problem by making a single reset-both error weigh up to 140 times the average instance. The effect is substantial: drop recovers to an F1 of 0.473 and reset-both becomes detectable, raising macro F1 by more than 11 points. The trade-off is a controlled degradation of the majority classes and of accuracy, while the weighted F1 is preserved. The persistent difficulty with reset-both, which reaches high recall but very low precision, has two root causes. First, the extreme imbalance leaves only 812 training examples, which is insufficient for GP to learn a precise pattern in a 21-dimensional space. Second, drop and deny share structural attributes (both are blocking actions), creating feature overlap that limits separability regardless of the fitness weighting. Promising directions for future work include oversampling techniques such as SMOTE, multi-objective fitness that jointly optimizes precision and recall, and stream-aware classifiers that explicitly model the minority distribution over time [Spinosa et al. 2009, Pimentel et al. 2014].

4.3. Concept Drift Stability

The absence of performance degradation at the DS1-to-DS2 boundary holds for both variants, and the weighted model even improves on DS2. One plausible explanation is that structural traffic attributes such as IP protocol, network zones, and application categories are relatively stable in educational institutions: the types of applications in use change slowly even if IP addresses and port numbers vary from day to day. Combined with the proportional split, which exposes the model to both phases during training, the learned expressions remain valid across the 15-day gap. This is consistent with the concept-drift literature, which distinguishes gradual from abrupt drift [Gama et al. 2014]; the gap between DS1 and DS2 introduces at most a gradual drift that the static GP model tolerates

without any adaptation mechanism.

4.4. Interpretability

A key advantage of Genetic Programming over black-box models such as neural networks and random forests is the interpretability of the evolved expressions. The resulting trees, ranging from 15 to 34 nodes, can be inspected to identify which attribute combinations drive each firewall decision, information that is potentially valuable for security engineers who wish to refine rules manually. The weighted-fitness trees are somewhat larger than the baseline ones, reflecting the additional complexity required to carve out decision regions for the rare classes.

5. Conclusion

This paper proposed and evaluated a Genetic Programming classifier for firewall log data in a temporal data stream scenario. The model was trained on a proportional 30% sample of a stream of 1.5 million network sessions and evaluated on the remaining 70% across 50 chronological windows and separately on each collection phase.

The results show that GP learns compact and effective classifiers for the majority classes, achieving a baseline F1 of 0.991 for allow and 0.910 for deny, with an overall weighted F1 of 0.888, comparable to traditional supervised methods reported in the related literature [Aljabri et al. 2022, Sharma et al. 2021]. The baseline, however, ignores the severely under-represented drop and reset-both classes. A class-weighted fitness recovers them, lifting macro F1 from 0.475 to 0.587 and drop F1 to 0.473 while preserving the weighted F1, at the cost of a moderate decline in majority-class accuracy. Both models proved stable across the concept-drift boundary, with the weighted variant even improving on the more recent data.

The vectorized fitness evaluation approach demonstrated that GP can be applied practically to industry-scale datasets, completing the full training of four OvR trees in under nine minutes for 455,000 instances and 21 attributes. Future work could combine the weighted fitness with resampling strategies and multi-objective optimization to improve minority-class precision, and incorporate novelty detection to identify rare patterns in the incoming flow [Spinosa et al. 2009, Luo et al. 2020].

References

Al-amri, R., Murugesan, R. K., Man, M., Abdulateef, A. F., Al-Sharafi, M. A., and Alkahtani, A. A. (2021). A review of machine learning and deep learning techniques for anomaly detection in IoT data. *Applied Sciences*, 11(12):5320.

Aljabri, M., Alahmadi, A. A., Mohammad, R. M. A., Aboulnour, M., Alomari, D. M., and Almotiri, S. H. (2022). Classification of firewall log data using multiclass machine learning models. *Electronics*, 11(12):1851.

Ertam, F. and Kaya, M. (2018). Classification of firewall log files with multiclass support vector machine. In *2018 6th International Symposium on Digital Forensic and Security (ISDFS)*, pages 1–4. IEEE.

Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., and Gagne, C. (2012). DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13:2171–2175.

Gama, J. (2010). *Knowledge Discovery from Data Streams*. Chapman & Hall/CRC.

Gama, J., Zliobaite, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):44:1–44:37.

Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

Ling, C., Ling-Jun, Z., and Li, T. (2009). Stream data classification using improved fisher discriminate analysis. *Journal of Computers*, 4(3):208–214.

Luo, Y., Yin, L., Bai, W., and Mao, K. (2020). An appraisal of incremental learning methods. *Entropy*, 22(11):1190.

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill, Inc., 1st edition.

Pimentel, M. A., Clifton, D. A., Clifton, L., and Tarassenko, L. (2014). A review of novelty detection. *Signal Processing*, 99:215–249.

Schindler, T. (2017). Anomaly detection in log data using graph databases and machine learning to defend advanced persistent threats. In *INFORMATIK 2017*.

Sharma, D., Wason, V., and Johri, P. (2021). Optimized classification of firewall log data using heterogeneous ensemble techniques. In *Proceedings of the IEEE International Conference*, pages 368–372. IEEE.

Spinosa, E. J., de Carvalho, A. P. D. L. F., and Gama, J. (2009). Novelty detection with application to data streams. *Intelligent Data Analysis*, 13(3):405–422.

Tu, Y. (2019). Machine learning. In *EEG Signal Processing and Feature Extraction*. Springer, Singapore.